# abaplint.app

**Documentation**

## Table of Contents


1. Introduction ..... 1
    1.1. Requirements ..... 1
2. Quick Start ..... 1
3. Configuration ..... 2
    3.1. Repository Access ..... 2
    3.2. Rule Configuration ..... 2
    3.3. App Configuration ..... 3
        3.3.1. Multiple Configurations ..... 3
        3.3.2. Experimental Features ..... 3
    3.4. Dependent GitHub Actions ..... 4
4. Annotations ..... 5
5. Observations ..... 6
    5.1. New void type used ..... 6
    5.2. DB field changed ..... 6
6. Suggestions ..... 8
7. Insights ..... 9
    7.1. Issues ..... 9
    7.2. Method Length ..... 9
    7.3. Statement Compatibility ..... 9
    7.4. Void Types ..... 10
    7.5. Dependencies ..... 10
    7.6. Package Coupling ..... 10
    7.7. UML Class Diagrams ..... 10
    7.8. Disabled Rules ..... 11
    7.9. Repository Relations ..... 11
8. Cross Check ..... 12
    8.1. Example ..... 12
9. ATC/SCI Integration ..... 14

| | |
|---|---|
| Home | https://github.com/heliconialabs/docs.abaplint.app |
| License | Creative Commons |
| Build | 2022-05-03 05:48:08 UTC |

# 1. Introduction

abaplint.app is a cloud based tool for helping ABAP developers develop ABAP. The app can be used for static analysis and continuous integration, providing multiple configurable rules to support any development guidelines.

## 1.1. Requirements

A GitHub user or organization is required for use with the abaplint.app. The latest version of abapGit is required for serializing the ABAP artifacts, abapGit works on SAP_BASIS 702 and up.

# 2. Quick Start

1. Create GitHub User or Organization
2. Install the app from the marketplace
3. Install abapGit in the ABAP system
4. Push ABAP code to GitHub via abapGit

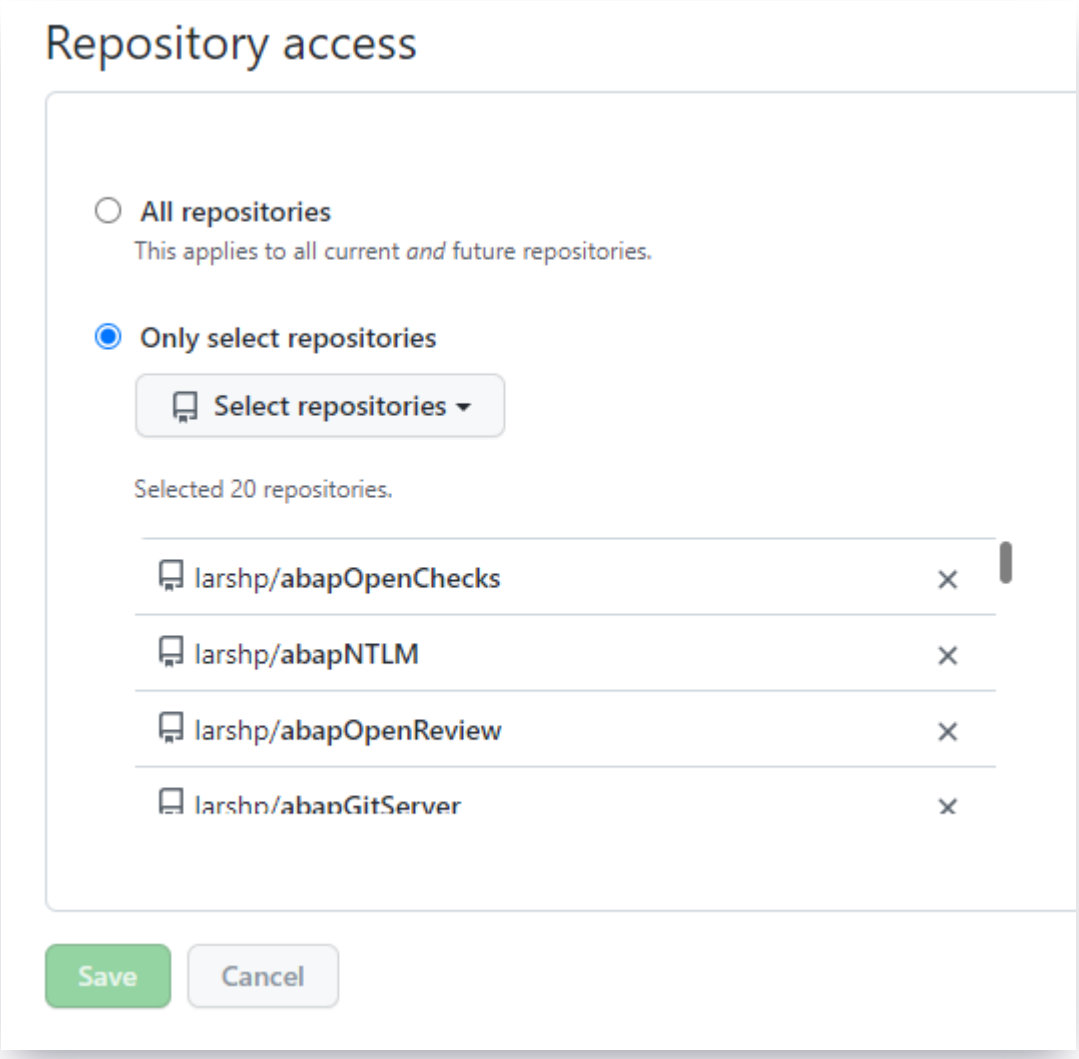
abaplint.app is automatically triggered for each commit or when a pull request is opened.

# 3. Configuration

## 3.1. Repository Access

The administrator of a GitHub organization controls which repositories that abaplint.app can access via https://github.com/apps/abaplint/installations/new.

Only grant access to repositories where abaplint.app will be used,




Administrators can revoke or grant additional repository access at any time, note that the app permissions are limited to read and updating information in pull requests.

## 3.2. Rule Configuration

abaplint is configured via `/abaplint.json` in the git repository, if no configuration file is present, the default configuration is used, which have all rules enabled.

The default configuration can be found on playground.abaplint.app, or by running `abaplint -d`on command line.

If editing the configuration file via vscode abaplint extension, the editor will know the format of the json configuration file and help discovering and updating rule configurations.

rules.abaplint.org lists and documents all rules which can be configured.

## 3.3. App Configuration

A file with one of the following filenames can optionally be placed in the root of the git directory,

- `abaplint-app.json`
- `abaplint-app.jsonc`
- `abaplint-app.json5`

Only the first file found will be taken into account.

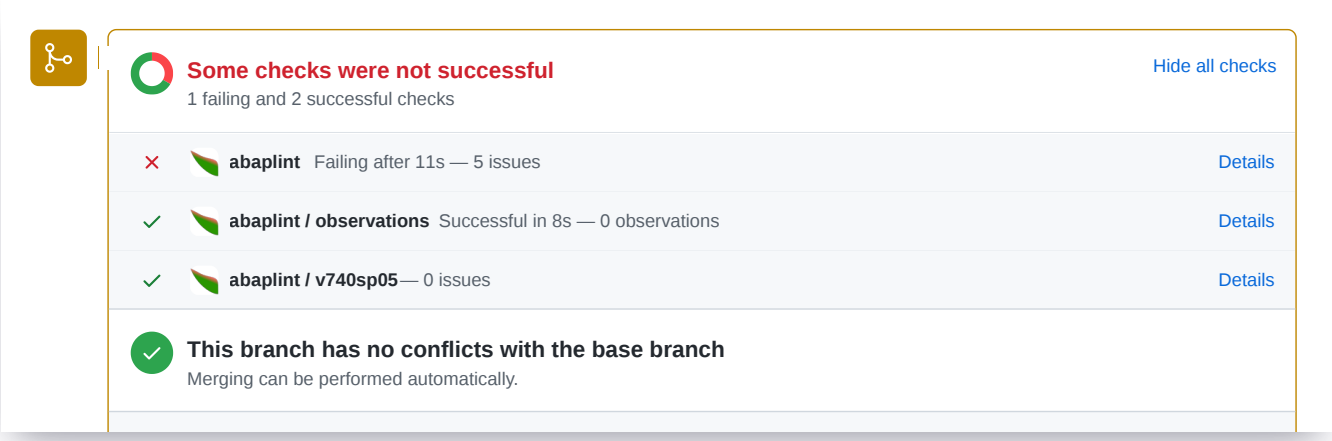
The file can be used to configure abaplint.app features. The corresponding schema can be accessed at https://schema.abaplint.app/schema.json.

### 3.3.1. Multiple Configurations

By adding a `abaplint-app.json` file it is possible to have abaplint.app run multiple abaplint configurations. Observations and suggestions are reported only for the first configuration. Configurations are processed in the sequence as defined in the file,

```
{
  "configurations": {
    "default": {
      "filename": "./abaplint.jsonc"
    },
    "v740sp05": {
      "filename": "./abaplintonprem.jsonc"
    }
  }
}
```

When running with the above configuration, abaplint.app also analyzes the source code using `abaplintonprem.jsonc` and issues the result as "abaplint / v740sp05",




### 3.3.2. Experimental Features

Experimental features are disabled by default, but can be enabled by setting

```
{
  "experimentalFeatures": true
}
```

These features change over time and are only documented when the feature becomes generally available.

## 3.4. Dependent GitHub Actions

abaplint.app helps with static analysis, the next step in a CI pipeline is typically to run the unit tests. Github Actions can be used to trigger any logic after the static analysis completes.

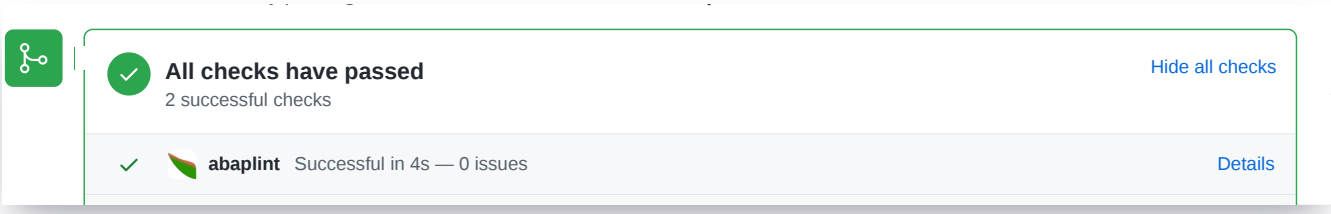
Template and working example can be found at https://github.com/heliconialabs/abaplint-app-dependent-action

Example Github Action configuration,

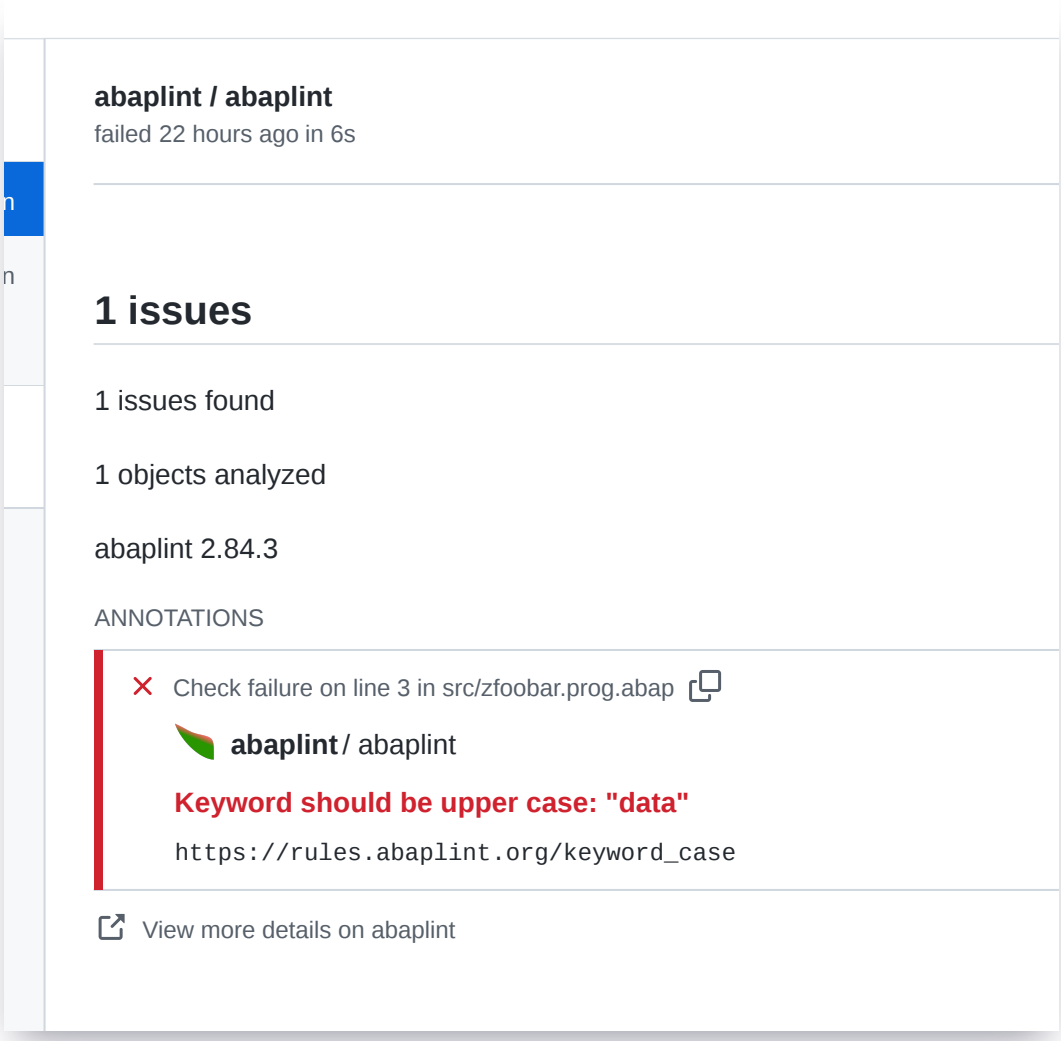
```
name: unit

on:
  pull_request

jobs:
  wait:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
     - name: Wait for other checks to succeed
       uses: lewagon/wait-on-check-action@v1.1.1
       with:
         ref: ${{ github.event.pull_request.head.sha }}
         check-name: abaplint
         repo-token: ${{ secrets.GITHUB_TOKEN }}
         wait-interval: 1
  unit:
    needs: wait
    runs-on: ubuntu-latest
    steps:
    - name: Echo github context
      run: echo '${{ toJson(github.event) }}' || true
```

# 4. Annotations

After installation and configuration of abaplint.app, it will report findings for each commit.




If issues are found, the "Details" link can be clicked, and the developer can check the findings directly in the pull request,

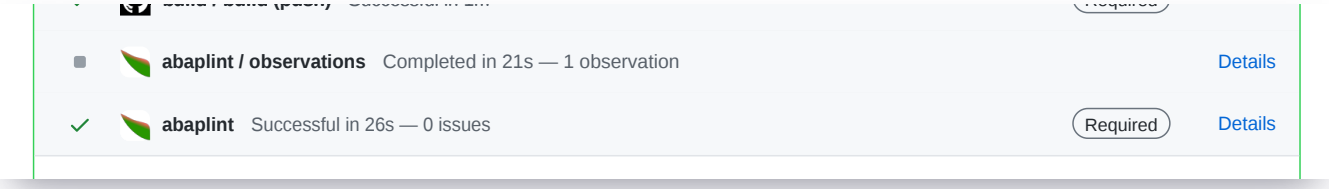



Annotations are updated after each push, and links directly to the branch which contains the change.

Additional information regarding the specific rules can be looked up on https://rules.abaplint.org

# 5. Observations

Observations are run only in the context of a pull request, and reports only "success" or "neutral" run status, so it will not block merges in case required checks are setup for branch protection.




If there are zero observations the step reports success, if some are found it reports the neutral status. A neutral status is shown in the pull request, which help s the reviewer to identify and check the observations.

For running examples see pull requests for the following repositories, https://github.com/heliconialabs?q=observation

## 5.1. New void type used

If abaplint is configured with `errorNamespace` some types are trusted by abaplint to be correct in all uses, these types are called void types.

This observation reports findings if new void types are introduced by the pull request, this helps identifying if the footprint towards eg. dependencies or the standard system increases.

## 5.2. DB field changed

Changing structures, data elements or domains can cause transparent database tables to be changed, this observation helps making these changes visible to reviewers.

This observation is reported if:

- If the type of a field is changed
- If a field is added
- If a field is removed

If a key field is added

- If a key field is removed

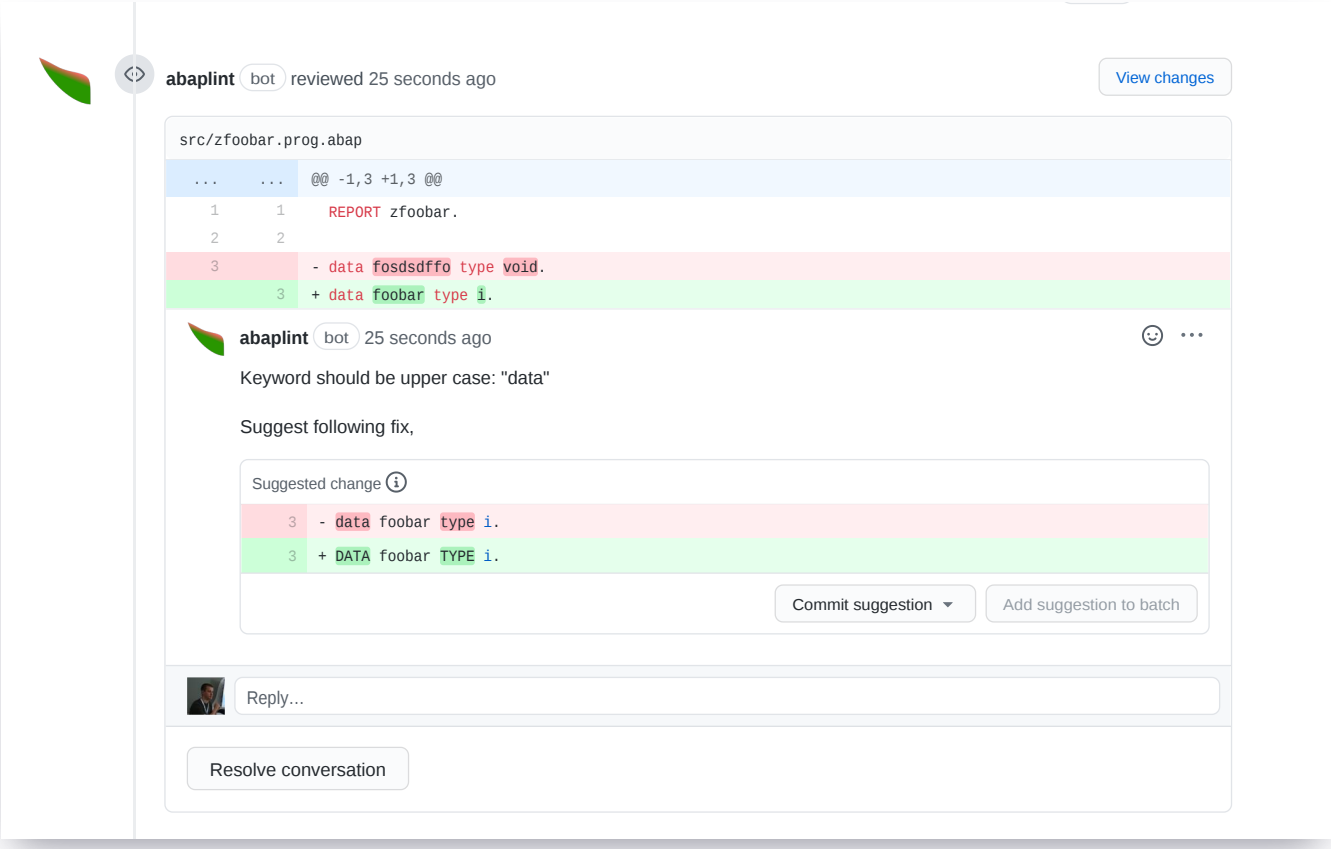
## 2 observations

| No | Description |
|---|---|
| 1 | Database field type changed, `ZTABL1-FIELD2`, `c LENGTH 1` -> `c LENGTH 2` |
| 2 | Database field added, `ZTABL1-NEW_FIELD` |

Base: `11f5992`, Head: `85de0be`

# 6. Suggestions

Suggestions by abaplint.app appear in pull requests, rules which have quick-fixes must be enabled, and only one suggestion appear at a time.

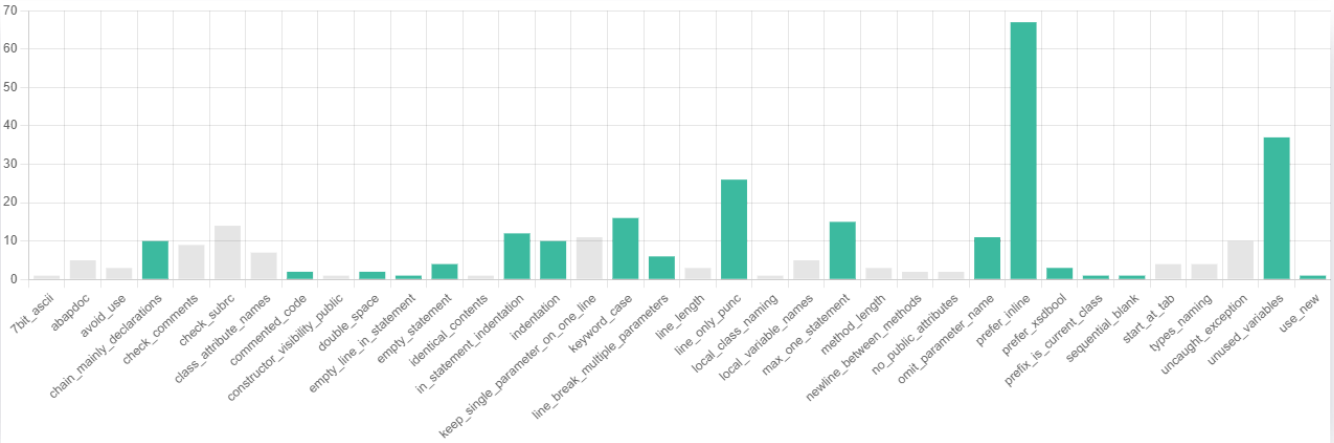


# 7. Insights

Various code insights are automatically generated daily by abaplint.app, insights can be accessed via web. Insights for public projects are public, users must login to access insights for private projects. All insights are generated only for the default branch.

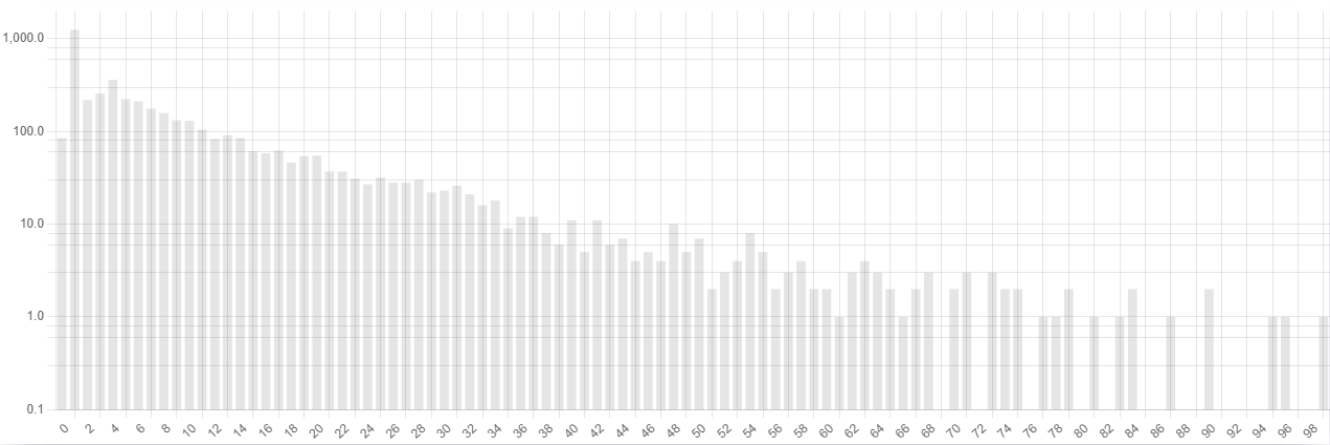
## 7.1. Issues

abaplint is run for the source code, if no configuration is present, the default configuration is used. Issues are listed, plus a graph gives an overview, issues with quick-fixes are shown in green,



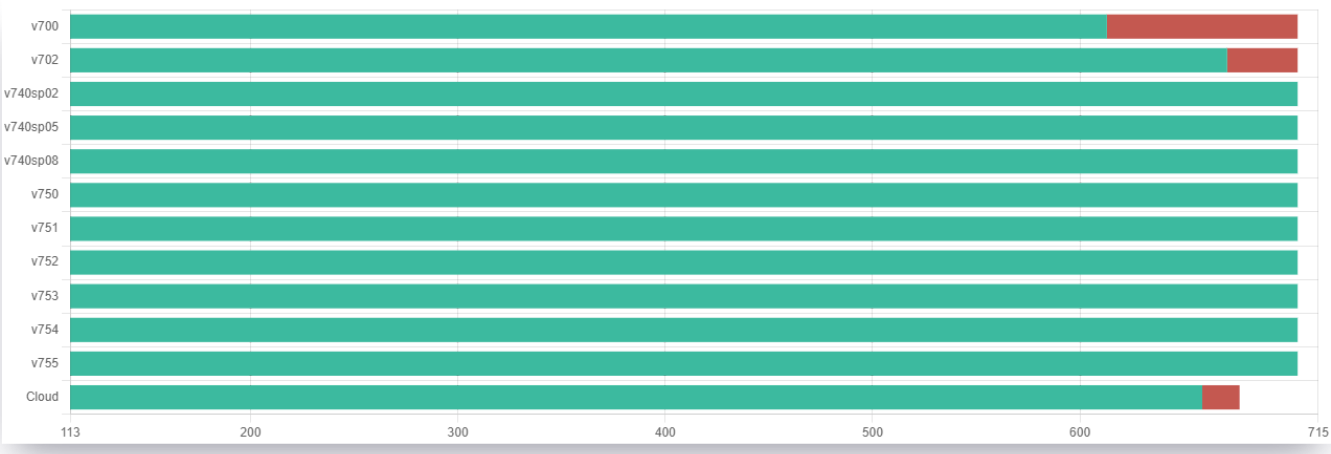

## 7.2. Method Length

Method length gives an overview of the method implementation lengths across the source code. Note that the scale is logarithmic,




## 7.3. Statement Compatibility

Statement parsing id performed against different language versions, this gives an overview of effort required to downport or upport the codebase.




## 7.4. Void Types

Lists void types and locations.

## 7.5. Dependencies

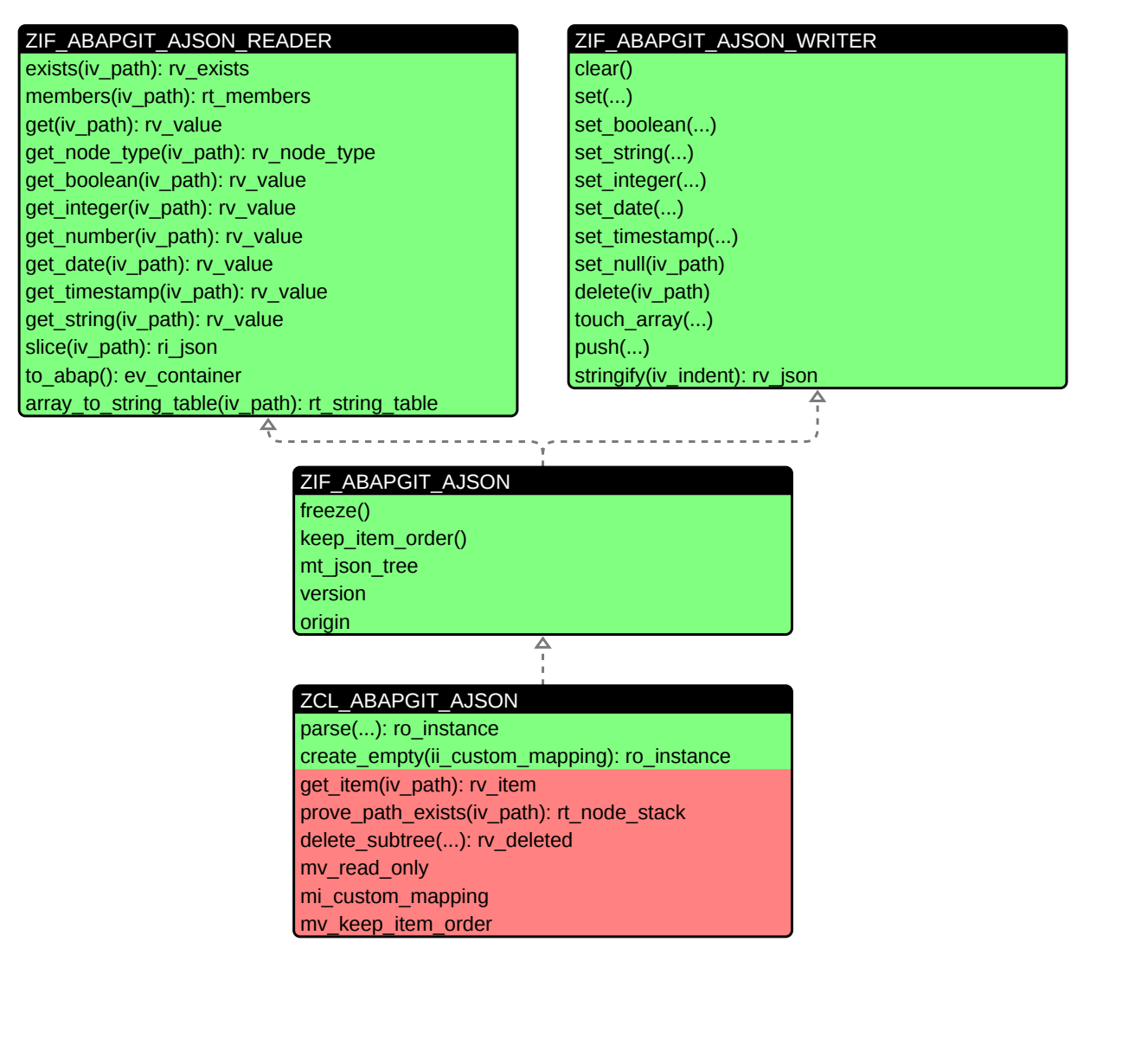
Lists where dependencies are sued in the codebase. Dependencies are configured in the `abaplint.json` file

## 7.6. Package Coupling

Work in progress

## 7.7. UML Class Diagrams

Provides auto generated UML diagrams for all global classes and interfaces, example:




## 7.8. Disabled Rules

If rules are disabled in the configuration, abaplint.app will try to enable to rule and provide the issue count and how much is quick-fixable.

## 7.9. Repository Relations

Dependencies from the abaplint configuration file is listed,

# Repo Relations

Based on the dependencies defined in the default abaplint configuration file.

| | |
|---|---|
| https://github.com/abapGit/abapGit | dependency |
| https://github.com/abaplint/deps | dependency |
| https://github.com/larshp/ABAP-Swagger | |

# 8. Cross Check

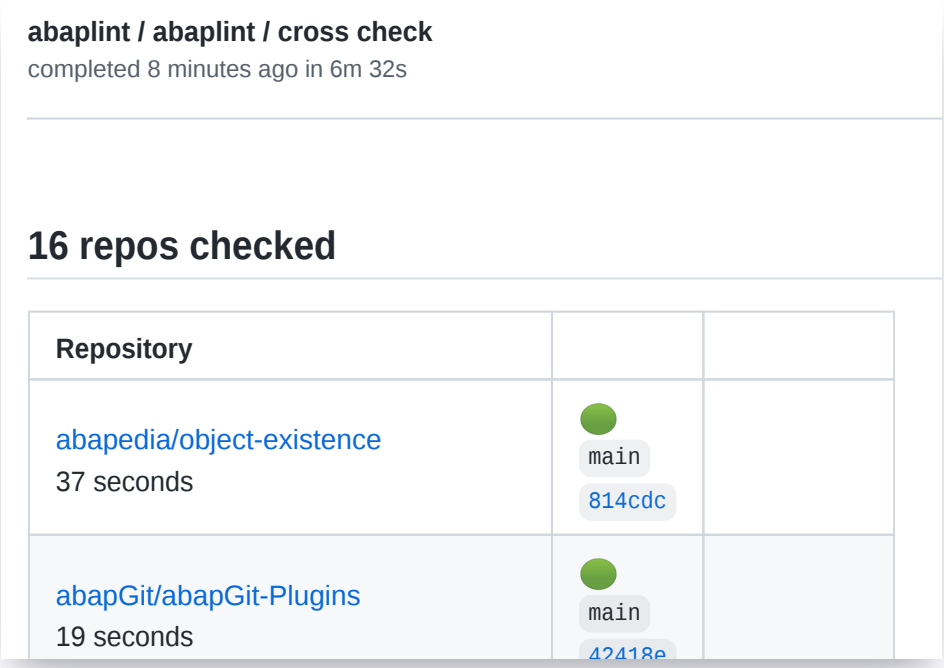
Some applications consists of multiple components maintained by different teams. This abaplint.app feature allows to discover syntax errors across repositories before changes are merged, so that fixes can be prepared in due time.

All repository relations and branches are found and checked one by one. If the checked already have errors, it is skipped.

Public repositories consider only public dependencies, private will take repositories in the same organization into account.

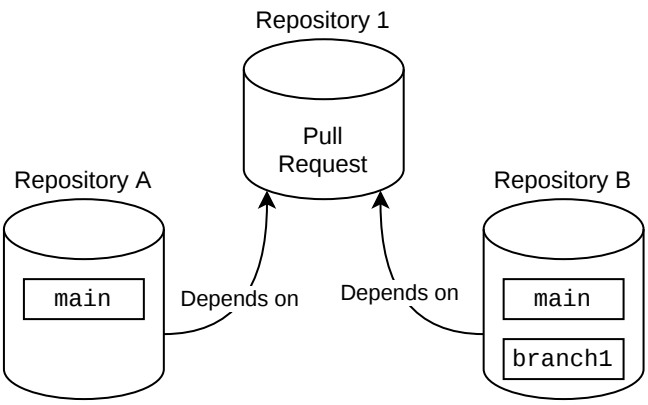
The cross check feature is enabled by default, but only reported if there are any related repositories, the feature can be disabled in `abaplint-app.json` if needed.

If there are no errors found, success is reported as a status check on the pull request. In case of new errors, a neutral status is reported, click the link from the status check to see the overview of the results,



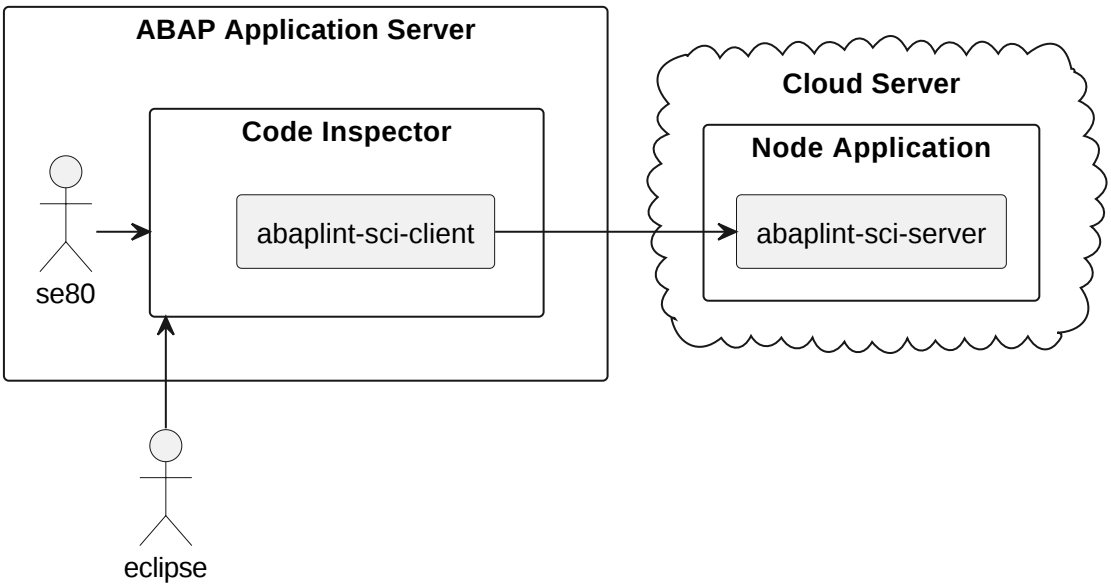

## 8.1. Example

A pull request is opened in "Repository 1", which have two dependent repositories: A and B,



The pull request will cross check all combinations,

- Repository A, branch `main` against the pull request
- Repository B, branch `main` against the pull request
- Repository B, branch `branch1` against the pull request

# 9. ATC/SCI Integration

The ABAP developer can check the working copy in the ABAP system via abaplint-sci-client